

Python's Object Model and Mutability

In Python, **what a variable is** and **what it points to** both matters.

1. Everything is an Object

In Python, everything (integers, strings, functions, even types themselves) is an **object**.

An object has:

1. An **Identity** (its address in memory).
2. A **Type** (e.g., integer, list).
3. A **Value** (the data it holds).

The object model is at the **core of how Python works**. If you're curious, look up chapter 6 and 7 of Ramalho's book. This is not required for this course, but life is boring if you only read what's in the syllabus.

Ramalho L. 2022. Fluent Python: Clear, Concise, and Effective Programming, Second Edition. Shroff/O'Reilly. 1012 p.

```
In [...] # x = [1, 2, 3]
          x = 1

          print(f"Identity (Memory Address): {id(x)}")
          print(f"Type: {type(x)}")
          print(f"Value: {x}")
```

```
Identity (Memory Address): 4369722280
Type: <class 'int'>
Value: 1
```

2. Mutability: Mutable vs Immutable

Type	Category	Behavior
Immutable	int, float, str, tuple	Cannot be changed after creation. Changes create a new object.

Type	Category	Behavior
Mutable	list, dict, set	Can be changed in-place without changing the identity.

Why Lists behave differently from Strings/Tuples

When you modify a list, you are changing the contents of that specific memory address. When you "modify" a string, Python actually creates a brand new string and reassigns your variable name to it.

```
In [... # Immutable Example: Strings
s = "Hello"
old_id = id(s)
s = s + " World"
print(old_id, id(s))
print(f"String ID changed: {old_id != id(s)}") # True

# Mutable Example: Lists
l = [1, 2]
old_id = id(l)
l.append(3)
print(old_id, id(l))
print(f"List ID changed: {old_id != id(l)}") # False

4725503728 4725992368
String ID changed: True
4725992192 4725992192
List ID changed: False
```

Functions vs Methods

- **Functions:** Standalone blocks of code called by name (e.g., `print(x)` or `len(x)`).
- **Methods:** Functions that "belong" to an object and are called using dot notation (e.g., `list.append()` or `string.upper()`).

```
In [... my_list = [3, 1, 2]
print(type(my_list))
# len() is a function
```

```
print(len(my_list))

# .sort() is a method belonging to the list object
sorted(my_list)
my_list.sort()
print(my_list)

str_example = 'vamsi Is a good person'
str_example.lower()
str_example.count('v')
```

```
<class 'list'>
```

```
3
```

```
Out[... [1, 2, 3]
```

```
[1, 2, 3]
```

```
Out[... 'vamsi is a good person'
```

```
Out[... 1
```

```
In [... # a quick example of how you can use python methods
# here we want to compute the percentage of different
```

```
insulin = 'MALWMRLLPLLALLLWGPDPAAAFVNQHLCGSHLVEALYLIV
```

```
amino_acids_1_letter = [
    "A", # Alanine
    "R", # Arginine
    "N", # Asparagine
    "D", # Aspartic acid
    "C", # Cysteine
    "E", # Glutamic acid
    "Q", # Glutamine
    "G", # Glycine
    "H", # Histidine
    "I", # Isoleucine
    "L", # Leucine
    "K", # Lysine
    "M", # Methionine
    "F", # Phenylalanine
    "P", # Proline
    "S", # Serine
    "T", # Threonine
    "W", # Tryptophan
    "Y", # Tyrosine
```

```

    "V"    # Valine
]

len_insulin = len(insulin)
print(len_insulin)

percentage_count = {}

for amino_acid in amino_acids_1_letter:
    # print(amino_acid)
    percentage_count[amino_acid] = (insulin.count(ami
    # break
# percentage_count
percentage_count.keys()
percentage_count.values()
import matplotlib.pyplot as plt
plt.bar(percentage_count.keys(),percentage_count.valu

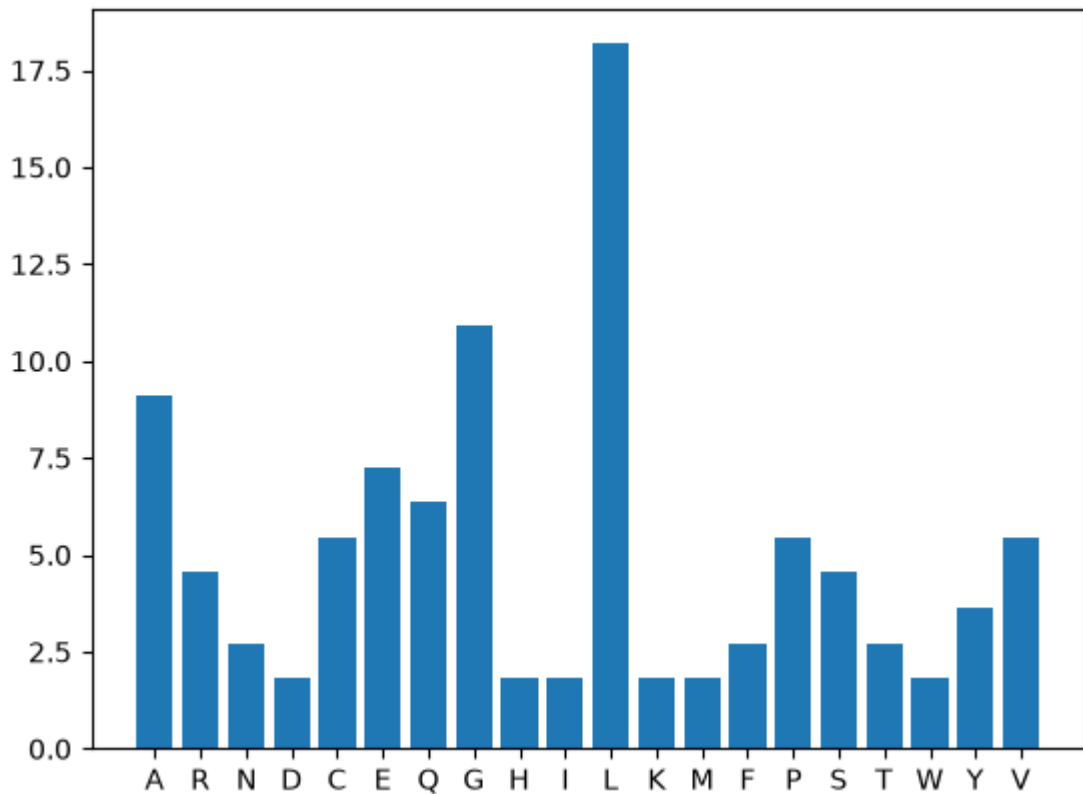
```

110

Out[...] dict_keys(['A', 'R', 'N', 'D', 'C', 'E', 'Q', 'G', 'H', 'I', 'L', 'K', 'M', 'F', 'P', 'S', 'T', 'W', 'Y', 'V'])

Out[...] dict_values([9.090909090909092, 4.545454545454546, 2.727272727272727, 1.8181818181818181, 5.454545454545454, 7.2727272727272725, 6.363636363636363, 10.909090909090908, 1.8181818181818181, 1.8181818181818181, 18.181818181818183, 1.8181818181818181, 1.8181818181818181, 2.727272727272727, 5.454545454545454, 4.545454545454546, 2.727272727272727, 1.8181818181818181, 3.6363636363636362, 5.454545454545454])

Out[...] <BarContainer object of 20 artists>



Reading material:

- Jake Vanderplus has written why Python has slow and what connections it has to do with Python's objects: <https://jakevdp.github.io/blog/2014/05/09/why-python-is-slow/>
- We will not discuss classes and object-oriented programming in this course. If you want to know about it, I recommend Realpython's tutorial: <https://realpython.com/python-classes/>